# Kagan

## Roadmap and architecture

A UNIX-like operating system for i386, RV32 and AArch64: design decisions, development phases, release milestones and risks

## Contents


**1 Introduction** **1**
- 1.1 Design goals . . . 1
- 1.2 The document set . . . 2
- 1.3 Development process . . . 2

**2 References** **2**
- 2.1 Editions . . . 2
- 2.2 Specifications and other references . . . 4

**3 Key technical decisions** **4**

**4 The phase map** **6**

**5 Phase summary** **8**

**6 What Kagan 1.0 and 2.0 look like** **9**

**7 Risks and mitigations** **10**

**8 Exit criteria** **11**


## 1 Introduction

Kagan is a small UNIX-like operating system written from scratch in C and assembly. Release 1.0 targets 32-bit x86 PCs: it boots with GRUB, runs preemptive multitasking with a separate address space per process, loads ELF programs from its own on-disk file system, and provides a POSIX-flavoured system call interface, a C library and a shell with pipes and redirection. Release 2.0 moves everything that depends on the processor behind one architecture interface and runs the same kernel, C library and file-system image on 32-bit RISC-V and 64-bit ARM. Every target runs in QEMU first; real hardware follows once the emulated target is stable.

This document is the entry point to the design documentation. It records the technical decisions that hold for the whole project, the nineteen development phases and the order between them, the architecture of releases 1.0 and 2.0, the project risks, and the exit criteria of every phase.

### 1.1 Design goals

**Small and readable.** A monolithic kernel with a modular source tree and a short path from a system call to the code that serves it. Assembly only where C cannot work, well under 5 % of the code.

**Correct at the protection boundary.** Every user pointer goes through one file, every system call has a documented validation contract, and every claim about isolation is backed by a negative test or a fuzzer.

**Portable by construction.** Generic code never sees the CPU; firmware differences end at one boot-information structure filled from Multiboot or a devicetree.

**Compatible where it costs nothing.** Linux i386 system call numbers on every architecture, and the MINIX V3 on-disk format, which Linux's `mkfs.minix` and `fsck.minix` create and check.

**Tested at every step.** Each phase closes with exit criteria that run headless in QEMU and in continuous integration.

### 1.2 The document set

Each phase has its own design document in `docs/pdf/`, built from the LaTeX sources in `docs/phases/`. All of them share one structure: an at-a-glance box with the phase's dependencies and deliverables; *Scope and motivation*; *Background*, the concepts the implementation relies on; *References*, with chapter, section and page references weighted PRIMARY, SECONDARY or OPTIONAL; *Implementation*, a sequence of numbered tasks, each with an acceptance check marked *Verify*; *Testing and debugging*, a table of failure modes; *Exit criteria*; review questions; and *Extensions*. A *Prior art* box compares the design with MINIX 3 and Linux, and in Stage V with xv6-riscv and Linux. The Stage V documents also cite free specifications; their chips are explained in a second legend at the end of those documents and of this one.

### 1.3 Development process

Phases are implemented in order, because each one builds on everything before it. Work proceeds in small commits, each of which builds and boots. A task is complete only when its Verify check passes, and a phase is complete only when all of its exit criteria hold; the next phase does not start while any of them is still open. Design decisions, hypotheses and debugging notes are recorded in the engineering journal, `docs/journal.md`.

## 2 References

Seven texts underpin the design. OSTEP, OSC and MOS provide the theory; OSDI documents a complete small kernel, MINIX 3, which serves as the main prior-art comparison; LKD and LKP show how Linux implements each subsystem today; and LOB is a practical guide to x86 bring-up whose chapters map onto Phases 1–14.

### 2.1 Editions

LOB and OSTEP are freely available from their authors; the others are published textbooks. Table 2 lists the exact editions the reference tables cite.

**⚠ Page numbers belong to these editions**

The page ranges in the reference tables are the *printed* page numbers of the editions in Table 2. They do not match other editions, and a reflowable e-book has no fixed pages, so each reference also gives chapter and section numbers. For OSTEP this is the normal case: each free chapter PDF starts at page 1, and the current version is paginated differently from version 0.90. Chapters 1–43, and every section the documents cite, have the same numbers in both versions. One title changed: chapter 23, *The VAX/VMS Virtual Memory System*, is now *Complete Virtual Memory Systems*, with VAX/VMS in §23.1.

| Code | Source | Role in the design |
|---|---|---|
| LOB | The little book about OS development | Practical x86 bring-up reference; its 14 chapters follow the build order of Phases 1–14. It uses GRUB Legacy and Bochs; the phase documents give the GRUB 2 and QEMU equivalents. |
| OSTEP | Operating Systems: Three Easy Pieces | Primary theory reference for every phase. |
| OSC | Operating System Concepts, 10th ed. | Comprehensive reference; a second treatment of each topic. |
| MOS | Modern Operating Systems, 4th ed. | Hardware, I/O, clocks, disks, security and operating-system design (Ch. 12). |
| OSDI | Operating Systems: Design and Implementation, 3rd ed. | MINIX 3, a complete x86 kernel printed line by line: the main prior-art comparison. MINIX is a microkernel; Kagan is monolithic. |
| LKD | Linux Kernel Development, 3rd ed. | How Linux implements scheduling, interrupts and bottom halves, synchronisation, memory, the VFS and block I/O: the second prior-art comparison. |
| LKP | Linux Kernel Programming (2021) | Current Linux practice for logging, the buddy and slab allocators, the CFS scheduler and synchronisation. |

**Table 1:** The seven reference texts and their role in the design.

| Code | Edition cited | Availability |
|---|---|---|
| LOB | Erik Helin, Adam Renberg. *The little book about OS development*, 2015. | Free at `https://littleosbook.github.io`, as a web page or a PDF. Page numbers refer to the PDF. |
| OSTEP | Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, version 0.90, Arpaci-Dusseau Books, 2015. | Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`, one PDF per chapter, in the current version (1.10). |
| OSC | Abraham Silberschatz, Peter Baer Galvin, Greg Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. ISBN 978-1-119-32091-3. | Wiley. |
| MOS | Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. ISBN 978-0-13-359162-0. | Pearson. |
| OSDI | Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. ISBN 978-0-13-142938-3. | Pearson Prentice Hall (out of print; widely available second-hand). |
| LKD | Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. ISBN 978-0-672-32946-3. | Addison-Wesley. |
| LKP | Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. ISBN 978-1-78995-343-5. | Packt. |

**Table 2:** Editions cited in the design documents.

### 2.2 Specifications and other references

Exact hardware behaviour comes from specifications rather than textbooks. The **Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A** (System Programming Guide) is the authority for protection, segmentation, paging, interrupts and the TSS, chapters 2–7. The **Multiboot Specification 0.6.96** defines the contract between GRUB and the kernel, and the **NASM manual** the assembler syntax. The **OSDev Wiki** (`https://wiki.osdev.org`) documents nearly every device used here, with known pitfalls. MIT's **xv6** is a useful small UNIX-like comparison kernel.

Stage V leaves x86, so the Intel manual, Multiboot and NASM no longer apply there. Its documents cite four further free references, each with its own chip. XV6 is the xv6 book, RISC-V edition, read with the `xv6-riscv` source, the closest small kernel to the Phase 17 port. RISCV is the RISC-V Instruction Set Manual, Volume I (unprivileged) and Volume II (privileged architecture), plus the RISC-V Supervisor Binary Interface (SBI) specification. ARM is the Arm Architecture Reference Manual for A-profile (DDI 0487) together with Arm's *Learn the architecture* guides. SPEC covers the Devicetree and Virtio specifications and QEMU's documentation for its `virt` boards. The port legend at the end of this document gives full titles and links.

## 3 Key technical decisions

These decisions hold for the whole project. The first eight follow the reference texts, so the design can be compared with MINIX 3 and Linux at every step; the last five cover the Stage V ports and follow the free specifications.

| Decision | Choice | Why |
|---|---|---|
| Target CPU | IA-32 (i386), 32-bit protected mode, BIOS boot | LOB, OSDI (MINIX 3) and the Intel examples in OSC §9.6 are all 32-bit x86. Two-level paging is the simplest real MMU to learn. A 64-bit x86 port stays a Phase 15 track; the RISC-V and ARM ports are Stage V. |
| Boot loader | GRUB via Multiboot 1; `qemu -kernel` for quick runs | A custom boot loader adds no value at this scale. Phase 1 implements a 512-byte boot sector to establish the boot path, then GRUB takes over. |
| Languages | C (gnu11) plus NASM (Intel syntax) | Assembly only where C cannot work: the entry point, descriptor table loads, interrupt stubs, context switch and entering user mode. That is under 5 % of the code. |
| Toolchain | `i686-elf` cross GCC and binutils, GNU Make, Git | A cross compiler never assumes a Linux target, which prevents a family of confusing bugs (Phase 0). |
| Emulator, debugger | QEMU and GDB (remote stub); Bochs is optional | QEMU is fast, scriptable, and has a monitor for inspecting registers and page tables. LOB uses Bochs, whose debugger is still handy for descriptor-table bugs. |
| Kernel structure | Monolithic, modular source tree | Simplest to get right for a first kernel. MINIX's microkernel is studied as a contrast, with an optional experiment in Phase 15. |
| On-disk file system | MINIX V3 layout with 1 KiB blocks, named KaganFS | The format described in OSDI §5.6, with source. Linux's `mkfs.minix -3` and `fsck.minix` create and check the images, and Linux can even mount them. |
| Testing | In-kernel `ktest` suites, headless QEMU, `isa-debug-exit`; host tests and parser fuzzing under AddressSanitizer and UBSan; continuous integration | `make test` boots, runs every test and exits with a pass or fail code, so regressions show up at once (Phase 7). CI runs it on every push; negative tests ("evil programs") and a system call fuzzer check the protection boundary from Phases 10 and 11 on. |

| Decision | Choice | Why |
|---|---|---|
| Port targets | RV32 (`rv32imac`) with Sv32 paging on QEMU `virt`, then optionally the custom Timur-RV32IMC core; AArch64 (ARMv8-A) at EL1 with 39-bit virtual addresses on QEMU `virt`, then optionally a Raspberry Pi 4 | Sv32 is two-level 4 KiB paging like x86, so the RISC-V kernel can stay at `0xC0000000`. AArch64 then changes the instruction set and the word size together: 4 KiB pages, three-level tables, the kernel in the `TTBR1` high half (Phases 17, 18). |
| Port firmware and discovery | OpenSBI in M-mode (the kernel runs in S-mode); QEMU or the Pi firmware on ARM; a flattened devicetree instead of Multiboot | The firmware hands over a devicetree that says where RAM and devices are. `lib/fdt.c` reads it into `struct bootinfo`, so generic code never knows which firmware booted it (Phase 16). |
| System calls on the ports | The same Linux i386 numbers everywhere; `ecall` with the number in `a7`, `svc #0` with it in `x8` | One table and one libc for three architectures; only the trap instruction and the registers change. Linux itself uses a different, generic table on RISC-V and arm64 (`write` is 64 and there is no `fork`), so its headers are not a source for these numbers. |
| Port toolchain | `riscv32-elf` and `aarch64-elf` cross GCC and binutils; GNU `as` for port assembly | Built exactly like `i686-elf` in Phase 0, with the same versions (Task 16.1). NASM only knows x86, so it stays for the i386 code; port assembly lives in preprocessed `.S` files. |
| Port devices | 16550 or PL011 console, PLIC or GICv2, SBI or Arm generic timer, virtio-blk disk, `sifive_test` or semihosting for `make test` | The `virt` boards have no PS/2 keyboard, VGA text screen or IDE disk by default. One virtio-blk driver serves both ports. All three targets are little-endian and KaganFS uses fixed-width fields, so the same disk image works everywhere. |

**Table 3:** Technical decisions for Kagan.

**Why monolithic rather than a microkernel?**

In a monolithic kernel, drivers, the file system and the scheduler all run in ring 0 and call each other as plain C functions. In a microkernel like MINIX 3 they run as separate user-space processes that talk by message passing (OSDI §1.5.5, §2.5.1; MOS §1.7). Microkernels isolate faults better, but they require IPC, servers and a protocol for every operation before anything works. Plain function calls keep the path from a design to running code short and make the kernel easier to reason about. Every phase still compares the design with MINIX, and Phase 15 specifies an experiment that moves one driver into a user-space server to measure the trade-off.

**Security starts where its subject starts**

The security goal for release 1.0 is concrete: no unprivileged program and no malformed disk image may crash or take over the kernel, and every claim of that kind is backed by a test. Each practice begins in the phase where the thing it protects first exists. Phase 0 writes a short threat-model stub and verifies the toolchain downloads; Phase 2 turns every warning into an error; Phase 6 makes kernel code read-only; Phase 7 adds continuous integration and a build that traps on undefined behaviour; Phase 10 writes the real threat model, routes every user pointer through one small file and starts a table of how each system call checks its arguments; Phase 11 starts the system call fuzzer; Phase 13 treats every on-disk structure as untrusted input. Phase 15 then consolidates: it finishes the audit, adds users and permissions, and ships

a release whose build can be reproduced and checked. The model, its trust boundaries and what is out of scope (physical attackers, the boot chain, networks, malicious hardware, side channels) live in `docs/security/threat-model.md`.

**When the references disagree**

The reference texts span fifteen years, from OSDI (2006) to LKP (2021), and sometimes describe different ways to do the same job. The rule: adopt the newer practice when it also suits Kagan, and keep the older one when it is simpler and the newer one only pays off in a much larger kernel. So Kagan **adopts** printk-style log levels and a `dmesg` buffer (Phase 2), top and bottom halves for interrupts (Phase 4), the `0`/`-E` and `ERR_PTR` error convention (Phase 7), saturating `refcount_t` counters and a run-time lock-order check (Phase 9). It **stages** the buddy page allocator after the bitmap (Phase 5) and slab caches under `kmalloc` after the first-fit heap (Phase 7), so the simple version is in place and tested first. It offers a CFS-style **fair scheduler** instead of MLFQ as the optional scheduler (Phase 8). And it **keeps** the MINIX-style buffer cache rather than Linux's page cache (Phase 12), and leaves loadable modules to the optional Track G, because a small monolithic kernel gains little from either.

**Why port after 1.0, and why RISC-V before ARM?**

A complete system on one architecture comes first, and x86 is the architecture of the main references. Porting afterwards is not wasted effort: every place where x86 leaked into generic code shows up when it is compiled for another machine, and fixing those leaks is what makes the generic code genuinely generic. RISC-V comes first because RV32 keeps the 32-bit word size, Sv32 paging has the same two-level, 4 KiB shape as x86, so the kernel can stay at `0xC0000000`, and the privileged architecture is small enough to specify in full. It also runs on the custom Timur-RV32IMC core. AArch64 then changes the instruction set and the word size (LP64) at the same time and brings a richer exception and memory model. That is the larger step, which is why it comes second, and it prepares the ground for the 64-bit x86 extension of Phase 15.

## 4 The phase map

The nineteen phases fall into five stages. Arrows show the build order. Each phase depends on everything before it. The first four stages lead to Kagan 1.0; the fifth ports it and leads to Kagan 2.0.

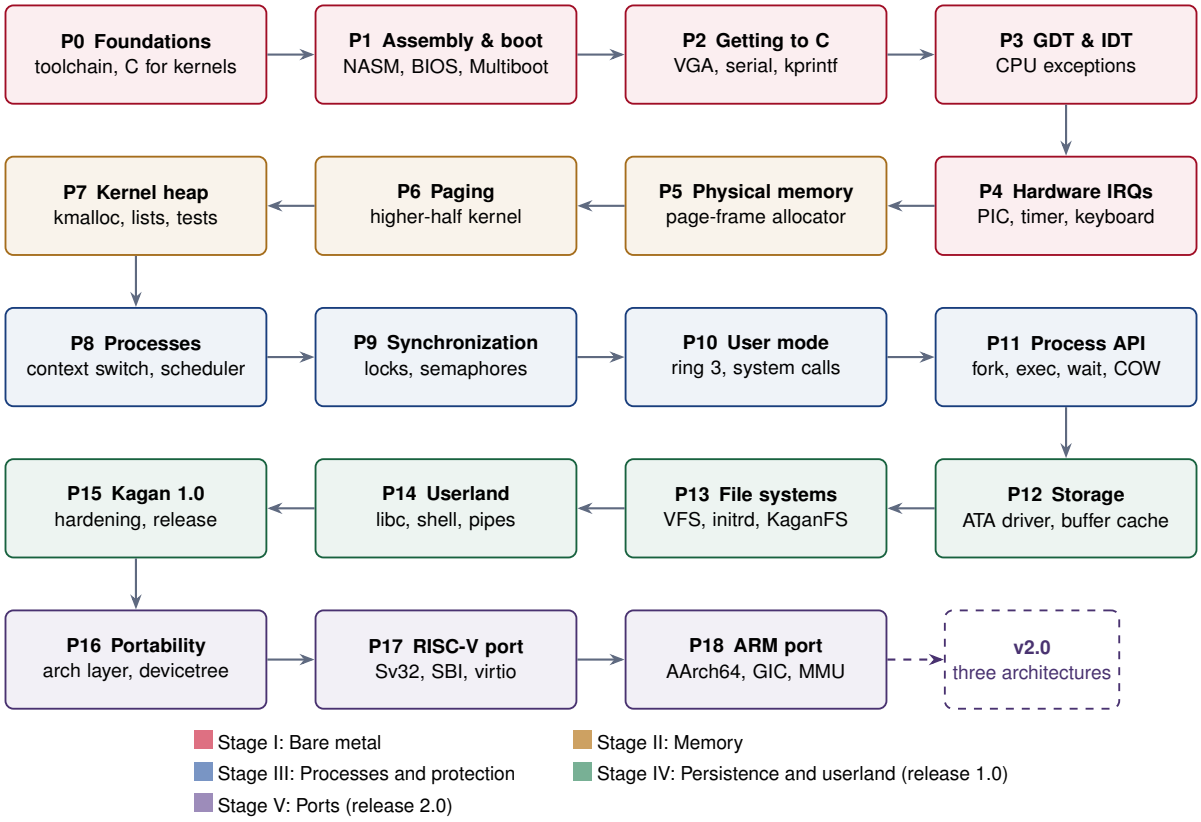



**Figure 1:** The nineteen phases, grouped into five stages.

**Stage I, Bare metal (Phases 0–4).** The toolchain, the boot path and the first hardware support. By the end the kernel runs C code, prints to the screen and the serial port, survives CPU exceptions with a readable error report, and reacts to the timer and the keyboard through a small built-in command monitor.

**Stage II, Memory (Phases 5–7).** The kernel learns which RAM exists and hands it out page by page, turns on paging, moves itself to the top gigabyte of the address space, and gets a `kmalloc` heap. The stage also establishes the automated test harness and continuous integration used for the rest of the project.

**Stage III, Processes and protection (Phases 8–11).** Multitasking arrives: kernel threads, context switches, a preemptive scheduler, then the locks that make concurrency safe. Programs move to ring 3 with their own address spaces and talk to the kernel through system calls. The stage ends with the UNIX process model: `fork` with copy-on-write, `exec`, `wait` and `exit`.

**Stage IV, Persistence and userland (Phases 12–15).** A disk driver and buffer cache provide storage, a VFS and KaganFS provide files, and a C library, shell and utilities complete a usable system. The last phase hardens, measures and documents it, and tags release 1.0; it also specifies the optional extensions.

**Stage V, Ports (Phases 16–18).** First the architecture layer is separated: everything x86 moves behind one interface, the generic code learns to compile for other machines, and the kernel learns to read a devicetree instead of Multiboot (16). Then Kagan is ported to 32-bit RISC-V on QEMU's `virt` board, with OpenSBI as the firmware, and optionally to the custom Timur-RV32IMC core (17). Finally it is ported to 64-bit ARM (AArch64) on QEMU's `virt` board, and optionally to a Raspberry Pi 4. The stage ends with Kagan 2.0: the same kernel, userland and KaganFS image on three architectures (18).

## 5 Phase summary

| Phase | Title | Deliverables |
|---|---|---|
| 0 | Foundations and toolchain | Cross compiler, QEMU, GDB and a Git repo; C warm-up library with unit tests; verified toolchain sources; threat-model stub |
| 1 | x86 assembly and booting | A 512-byte boot sector; a Multiboot kernel that GRUB and QEMU boot |
| 2 | Getting to C: screen, serial, kprintf | VGA text driver, serial logging, `kprintf`, `panic` |
| 3 | Segmentation, IDT and exceptions | GDT and IDT; register dump on every CPU exception |
| 4 | Hardware interrupts | PIC, 100 Hz timer, keyboard driver, kernel command monitor |
| 5 | Physical memory management | Memory map parsing, bitmap then buddy page-frame allocator, first kernel tests |
| 6 | Paging and the higher-half kernel | Paging on, kernel at `0xC0000000`, VMM API, page-fault reports, read-only kernel code |
| 7 | Kernel heap and data structures | `kmalloc`/`kfree` on slab caches, intrusive lists, `make test` in headless QEMU, CI and a trap-on-UB build |
| 8 | Processes and scheduling | Kernel threads, context switch, preemptive round-robin (optional fair scheduler), sleep, FPU state per task |
| 9 | Synchronization | Spinlocks, wait queues, mutexes, semaphores, `refcount_t`, lock-order checking |
| 10 | User mode and system calls | TSS, ring 3, `int 0x80`, checked user-memory access, validating ELF loader, threat model, first user programs |
| 11 | Process API | `fork` (copy-on-write), `exec`, `wait`, `exit`, `brk`, basic signals, system call fuzzer |
| 12 | Storage | ATA PIO driver, block layer, MBR partitions, buffer cache |
| 13 | File systems | VFS, file descriptors, tar initrd, writable KaganFS that rejects corrupted images |
| 14 | Userland | Kagan libc, pipes, tty line discipline, `kash` shell, coreutils |
| 15 | Hardening and release | Final security audit, users, permissions and sessions, benchmarks, docs, a reproducible and signed v1.0; optional tracks |
| 16 | Portability: an architecture layer | x86 moved to `kernel/arch/i386`, `include/arch.h`, `struct bootinfo`, a host-tested devicetree parser, one Makefile for three `ARCH` values |
| 17 | The RISC-V port | Kagan on `qemu-system-riscv32 -machine virt`: OpenSBI, Sv32, PLIC, virtio-blk, boot to `kash`; optionally the Timur-RV32IMC core |
| 18 | The ARM port | Kagan on `qemu-system-aarch64 -machine virt`: EL1, GICv2, generic timer, MMU, LP64 userland; v2.0 on three architectures; optionally a Raspberry Pi 4 |

## 6 What Kagan 1.0 and 2.0 look like

Figure 2 shows the finished 1.0 system and the phase that builds each part. It is worth coming back to this picture as the map of where each piece fits. Figure 3 shows what Stage V changes: the generic kernel stays one body of code, and everything that depends on the processor or the board moves below `include/arch.h`, with one backend per architecture.

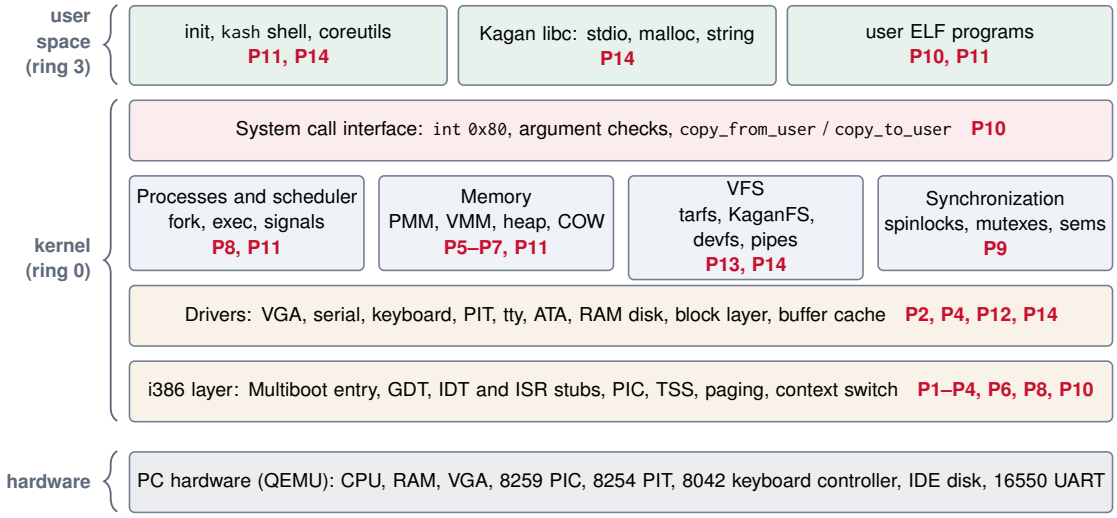



**Figure 2:** Layers of Kagan 1.0, with the phase that builds each part.

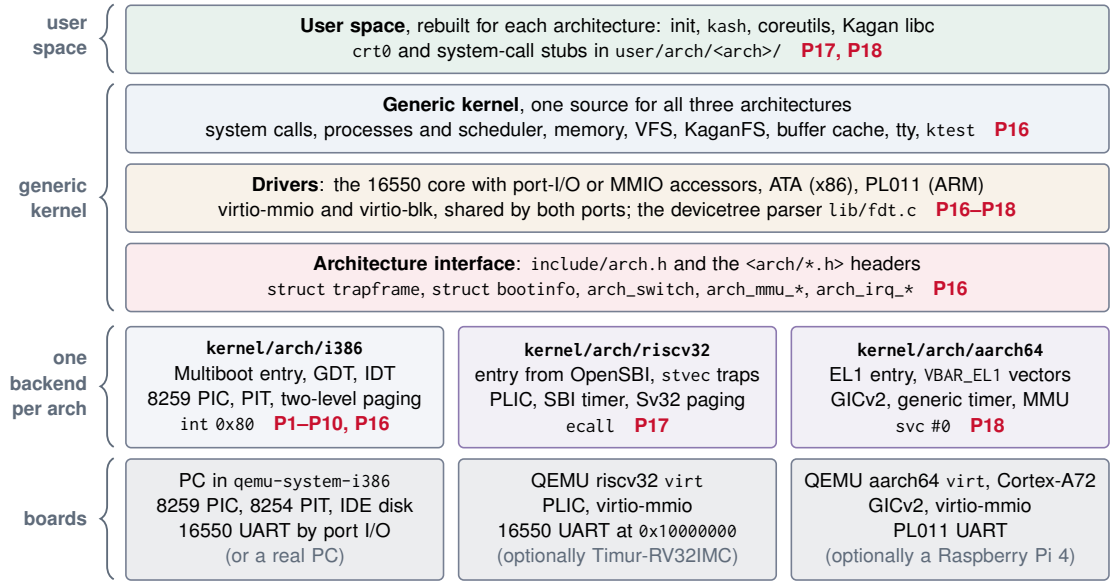



**Figure 3:** Kagan 2.0 after Stage V: one generic kernel, one architecture interface, three backends and three boards.

The source tree grows into the layout below; each phase document states which directories it adds. The entries marked `(Stage V)` arrive in Phases 16–18.

**kagan-os/ (layout at 1.0, plus the Stage V additions)**

```
kagan-os/
|-- docs/                 design documents (LaTeX), journal.md, security/
|   `-- porting.md        the leak audit and architecture-interface notes        (Stage V)
|-- boot/                 loader.s (Multiboot entry), grub.cfg
|-- kernel/
|   |-- arch/i386/        gdt.c idt.c isr.s irq.c pic.c pit.c tss.c paging.c switch.s
```

```
|   |   `-- include/arch/  the x86 headers behind the architecture interface      (Stage V)
|   |-- arch/riscv32/      boot.S entry.S trap.c sbi.c plic.c timer.c sv32.c      (Stage V)
|   |                      switch.S user.S arch.mk kernel.ld include/arch/
|   |-- arch/aarch64/      boot.S vectors.S trap.c gic.c timer.c mmu.c            (Stage V)
|   |                      switch.S user.S arch.mk kernel.ld include/arch/
|   |-- mm/                pmm.c vmm.c heap.c
|   |-- proc/              task.c sched.c fork.c exec.c signal.c
|   |-- sync/              spinlock.c mutex.c semaphore.c waitq.c
|   |-- fs/                vfs.c file.c tarfs.c kaganfs.c devfs.c pipe.c
|   |-- drivers/           vga.c serial.c keyboard.c tty.c ata.c ramdisk.c blockdev.c bcache.c
|   |                      + uart16550.c pl011.c virtio_mmio.c virtio_blk.c       (Stage V)
|   |-- syscall/           syscall.c uaccess.c
|   `-- kmain.c
|-- lib/                   libk: string.c printf.c bitmap.c ringbuf.c list.h
|                          + fdt.c, the devicetree parser                         (Stage V)
|-- include/               kernel headers
|                          + arch.h and bootinfo.h                                (Stage V)
|-- user/
|   |-- libc/              Kagan libc
|   |-- arch/<arch>/       crt0 and system call stubs for each architecture       (Stage V)
|   |-- init/ sh/ bin/     init, kash, coreutils
|   `-- user.ld
|-- tests/                 ktest suites
|   `-- data/              devicetree blobs dumped from QEMU, for the FDT tests   (Stage V)
|-- tools/                 mkkaganfs.c and helper scripts
|-- link.ld
`-- Makefile
```

## 7 Risks and mitigations

Kernel development has predictable failure modes, the worst of them a machine that silently reboots with no output. The table lists the project risks and their mitigations; the protocol after it is the standard debugging procedure.

| Risk | Likelihood | Mitigation |
|---|---|---|
| Triple faults (instant reboot) with no output | High | Serial logging from Phase 2 on; run QEMU with `-d int,cpu_reset -no-reboot`; GDB; small commits so `git bisect` can find the change that broke it. |
| Errors in hand-written assembly | High | Keep assembly to small stubs behind C interfaces, check it against compiler output, and single-step every stub once in GDB. |
| Memory corruption that shows up far from its cause | High | Assertions everywhere, magic numbers in headers, guard pages, `heap_check()`, deterministic tests. |
| A single defect blocks progress for days | High | Follow the debugging protocol below; time-box the investigation, then change approach. |
| Scope creep (GUI or networking too early) | Medium | Exit criteria are the definition of done. New ideas go on the extensions list in Phase 15. |
| A user program or a corrupted disk image can crash the kernel, found only at the end | Medium | Each phase that adds an entry point adds its negative tests at once: evil programs from Phase 10, the system call fuzzer from Phase 11, host fuzzing of every parser from Phase 13, all run in CI. |
| Toolchain trouble (cross-GCC build, Flatpak sandbox) | Medium | Phase 0 checklist, a host terminal, documented fallback compiler flags. |
| Outdated tooling in LOB | High | The phase docs give current equivalents (`grub2-mkrescue`, QEMU); the OSDev Wiki has errata. |

| Risk | Likelihood | Mitigation |
|---|---|---|
| Long single-maintainer project stalls | Medium | Every phase ends in a testable, demonstrable milestone; progress and decisions are recorded in the engineering journal. |
| Stage V: x86 habits carried into the ports (a forgotten `sfence.vma`, missing TLB maintenance or barriers) | High | Check every step against the specification, single-step in GDB, and compare with xv6-riscv or Linux. |
| Stage V: QEMU is more forgiving than real hardware (caches, alignment, uninitialised registers) | Medium | Keep real hardware optional and last: the QEMU milestone comes first and counts on its own. |
| Stage V: toolchain trouble with new targets (multilib, `-march` strings with `_zicsr`) | Medium | Section 9 of `docs/setup.md`, *Toolchains for the ports*, and the Phase 16 checklist (Task 16.1). |
| Stage V: the custom core lacks a feature Kagan needs (S-mode, an MMU, a timer) | Medium | Task 17.13's requirements checklist. The QEMU milestone stands on its own without the core. |
| Stage V: scope creep (SMP, graphics on the ports) | Medium | The Stage V exit criteria are the definition of done. |

**Debugging protocol**

1. **Reproduce it reliably**, then make it smaller: remove code until the shortest path to the bug remains.
2. **Look, don't guess.** Use the serial log, `-d int`, the QEMU monitor (`info registers`, `info mem`), and GDB breakpoints.
3. **Write the hypothesis down** in the engineering journal before changing code. Change one thing at a time.
4. **Re-read the section** of the Intel manual (in Stage V, the RISC-V or Arm specification) or the reference for the exact hardware rule involved.
5. **Compare with MINIX** (OSDI source) or xv6: how does a working kernel do the same step?
6. **After two sessions without progress**, escalate: write the problem up with the journal notes and ask on the OSDev forum or a relevant mailing list. Writing it up often exposes the cause.

## 8 Exit criteria

A phase is complete when every one of its exit criteria holds.

- ☐ **P0** `i686-elf-gcc`, NASM, QEMU and GDB work; repo created; toolchain signatures verified; C warm-up library and overflow helpers pass their tests; threat-model stub written.
- ☐ **P1** Boot sector prints a message; Multiboot kernel boots via `-kernel` and via a GRUB ISO.
- ☐ **P2** Colored banner on screen, same log on the serial port, `kprintf` and `panic` tested; the kernel builds with `-Werror`.
- ☐ **P3** Own GDT and IDT loaded; divide-by-zero shows a register dump; `int3` returns cleanly.
- ☐ **P4** Timer ticks at 100 Hz; keyboard input with shift and caps; monitor commands run.

- ☐ **P5** Memory map printed; bitmap and then buddy allocator pass the tests from 16 MiB to 3 GiB of RAM.
- ☐ **P6** Kernel runs at `0xC0000000`; null dereference produces a clean page-fault report; a write to kernel code faults.
- ☐ **P7** 10 000-operation stress test passes through the slab caches; `make test` runs headless and in CI; the trap-on-UB build passes; allocation failures are handled.
- ☐ **P8** Several kernel threads preempt each other; `sleep` works; no leaks over 1000 task cycles; FPU state stays with its task.
- ☐ **P9** Race demo fixed by locks; bounded buffer and dining philosophers run without deadlock; lock order checked.
- ☐ **P10** Ring-3 program prints through `write`; faulty and malicious programs are killed or get errors, the kernel survives; malformed ELF files are rejected; every system call has a row in the boundary table.
- ☐ **P11** `fork/exec/wait` chain works; copy-on-write verified, also through `copy_to_user`; no leaks over 1000 forks; ten minutes of fuzzing without a panic.
- ☐ **P12** QEMU disk identified; data survives reboot; partitions detected and corrupted tables rejected; buffer cache in use.
- ☐ **P13** Files written by Kagan pass `fsck.minix` on the host; file descriptor semantics tested; corrupted images are rejected without a panic; the parsers survive an hour of host fuzzing.
- ☐ **P14** Boots to the `kash` prompt; pipes, redirection and Ctrl+C work; coreutils present; resource limits enforced.
- ☐ **P15** One hour of fuzzing as an ordinary user without a panic; permission and session tests pass; benchmarks, docs and the threat model finished; two clean builds match and v1.0 is tagged and signed.
- ☐ **P16** Generic code free of x86 (audit in `docs/porting.md`); `make ARCH=i386 test` passes; generic code compiles for riscv32 and aarch64 without warnings; FDT parser passes its host tests.
- ☐ **P17** `qemu-system-riscv32 -machine virt` boots to the `kash` prompt; `make ARCH=riscv32 test` passes, evil programs included; W^X holds; a KaganFS image written on RISC-V passes `fsck.minix`.
- ☐ **P18** `qemu-system-aarch64 -machine virt` boots to the `kash` prompt; `make ARCH=aarch64 test` passes, evil programs included; W^X holds; the same KaganFS image works on all three; v2.0 tagged.

## References


LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.


LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.

## Port references (Stage V)


XV6 Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition. The companion book to MIT's xv6-riscv kernel. Free at https://pdos.csail.mit.edu/6.1810/; source at https://github.com/mit-pdos/xv6-riscv.

RISCV *The RISC-V Instruction Set Manual*, Volume I (Unprivileged ISA) and Volume II (Privileged Architecture); *RISC-V Supervisor Binary Interface Specification*. Free at https://riscv.org/specifications/ratified/ and https://github.com/riscv-non-isa/riscv-sbi-doc.

ARM *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487), and Arm's *Learn the architecture* guides: AArch64 exception model, AArch64 memory management, Generic Timer, GIC. Free at https://developer.arm.com/documentation.

SPEC *Devicetree Specification* (https://www.devicetree.org/specifications/); *Virtual I/O Device (VIRTIO)*, OASIS (https://docs.oasis-open.org/virtio/virtio/); the QEMU documentation for the RISC-V and Arm `virt` boards (https://www.qemu.org/docs/master/system/).


All four are free. Specifications are revised often, so the reference tables cite them by chapter and section title rather than by page.